[75.07 / 95.02 / TB025]

Algoritmos y programación III
Paradigmas y programación

Trabajo práctico 2: Catan

Estudiantes:

Nombre	Padrón	Mail

Tutor:

Nota Final:

Índice

1. Objetivo 3
2. Consigna general 3
3. Especificación de la aplicación a desarrollar 3
4. Interfaz gráfica 6
5. Herramientas 7
6. Entregables 8
7. Formas de entrega 8
8. Evaluación 8
9. Entregables para cada fecha de entrega 9
Entrega 0 (Semana 12 del calendario) 9
Entrega 1 (Semana 13 del calendario) 9
Entrega 2 (Semana 14 del calendario) 10
Entrega 3 (Semana 15 del calendario) 10
Entrega 4 (Semana 16 del calendario) 10
10. Informe 11
Supuestos 11
Diagramas de clases 11
Diagramas de secuencia 11
Diagrama de paquetes 11
Detalles de implementación 11
Excepciones 11
Anexo 0: ¿Cómo empezar? 12
Requisitos, análisis 12
Anexo I: Buenas prácticas en la interfaz gráfica 12
Prototipo 12
JavaFX 12
Recomendaciones visuales 13
Tamaño de elementos 13
Contraste 13
Uso del color 13
Tipografía 13
Recomendaciones de interacción 14
Manejo de errores 14
Confirmaciones 14
Visibilidad del estado y otros 14

1. Objetivo

Desarrollar una aplicación de manera grupal aplicando todos los conceptos vistos en el curso, utilizando un lenguaje de tipado estático (Java) con un diseño del modelo orientado a objetos y trabajando con las técnicas de TDD e Integración Continua.

2. Consigna general

Desarrollar la aplicación completa, incluyendo el modelo de clases, sonidos e interfaz gráfica. La aplicación deberá ser acompañada por pruebas unitarias e integrales y documentación de diseño.

3. Especificación de la aplicación a desarrollar

3.1. Objetivo General del Juego

El juego Catan es un juego de estrategia y gestión de recursos donde el objetivo principal es ser el primer jugador en alcanzar **10 Puntos de Victoria (PV)**. Los PV se obtienen al construir poblados, ciudades, y mediante cartas especiales ("**Gran Caballería**" y "**Gran Ruta Comercial**").

3.2. Componentes y Estructura del Tablero

El tablero es modular y se compone de **19 hexágonos de Terreno** rodeados por **Bordes de Mar** y **9 Puertos**.

Tipo de Componente	Descripción
Terrenos	Cada hexágono produce un tipo de **Recurso**: Madera (bosque), Ladrillo (colina), Lana (pastizal), Grano (campo) o Mineral (montaña).
Número de Fichas	**18 fichas numeradas** (2-12, excluyendo el 7 y con números repetidos) se colocan aleatoriamente sobre los Terrenos

	(uno por hexágono, excluyendo el Desierto).
Desierto	Un hexágono que **no produce recursos**. Es la ubicación inicial del **Ladrón**.

3.3. Jugadores y Estructura de Turno

El juego es para **3 o 4 jugadores** (la descripción asume 4, pero debe ser escalable).

A. Inicio del Juego (Setup)

1. **Colocación Inicial:** Cada jugador realiza **dos turnos de colocación** en orden.
    - **Primer Turno:** Cada jugador coloca **un Poblado** en un cruce (vértice) libre de tres hexágonos adyacentes y **una Carretera** adyacente a ese poblado. Reciben **una carta de recurso** por cada Terreno adyacente a su segundo poblado colocado.
    - **Segundo Turno (inverso):** Se repite el proceso, y esta vez los jugadores reciben **una carta de recurso** por cada Terreno adyacente a su **segundo poblado** que han colocado.

B. Estructura del Turno

El turno de cada jugador sigue la siguiente secuencia:

1. **Lanzamiento de Dados:** El jugador activo lanza **dos dados de 6 caras** para determinar la **producción de recursos**.
    - **Resultado 2-6 o 8-12:** Los Terrenos con la ficha de número coincidente producen el recurso asociado. Los jugadores con **Poblados** o **Ciudades** adyacentes a ese hexágono reciben las cartas de recurso correspondientes (1 carta por Poblado, 2 cartas por Ciudad).
    - **Resultado 7 (Ladrón): No hay producción de recursos.**
        - **Descarte:** Cualquier jugador con **más de 7 cartas de Recurso** debe descartar **la mitad** de ellas, redondeando **hacia abajo**.
        - **Movimiento del Ladrón:** El jugador activo **mueve el Ladrón** al hexágono de Terreno que elija.
        - **Robo:** El jugador activo **roba una carta de recurso al azar** de uno de los jugadores que tenga una construcción adyacente al nuevo hexágono ocupado por el Ladrón. El Terreno ocupado por el Ladrón **no produce recursos** hasta que el Ladrón se mueva de nuevo.
2. **Fase de Acción Comercial/Construcción:** El jugador activo puede realizar, en cualquier orden y tantas veces como sus recursos lo permitan:
    - **Comercio:**
        - **Comercio Interior:** Intercambiar cartas de recursos con otros jugadores (a tasas negociadas libremente).
        - **Comercio Marítimo (Banca):** Intercambiar recursos con la banca a las siguientes tasas:

        - **Tasa Estándar (4:1):** 4 cartas del mismo recurso por 1 carta de cualquier otro recurso.
        - **Puerto Específico (2:1):** 2 cartas del recurso indicado en un puerto por 1 carta de cualquier otro recurso.
        - **Puerto Genérico (3:1):** 3 cartas de cualquier recurso por 1 carta de cualquier otro recurso.
    - **Construcción (Gasto de Recursos):**

Construcción	Costo (Recursos)	Puntos de Victoria (PV)	Reglas de Colocación/Mejora
Carretera	1 Madera + 1 Ladrillo	0	Debe ser adyacente a otra de mis Carreteras o Poblados/Ciudades.
Poblado	1 Madera + 1 Ladrillo + 1 Lana + 1 Grano	**1 PV**	Debe estar conectado a una de mis Carreteras. Debe respetar la "**Regla de Distancia**" (No puede estar en un cruce adyacente a otro Poblado o Ciudad, propia o ajena).
Ciudad	2 Grano + 3 Mineral	**2 PV**	**Mejora** un Poblado existente. El Poblado es retirado y reemplazado por la Ciudad.
Carta de Desarrollo	1 Lana + 1 Grano + 1 Mineral	Varían	Se compran de un mazo oculto. Solo se puede jugar **una** carta de Desarrollo por turno (excepto las Cartas de PV).

3. Jugar Carta de Desarrollo (Opcional): El jugador puede jugar **una** Carta de Desarrollo (a menos que sea una Carta de PV, que no se juega, sino que se revela al final). No se puede jugar una Carta de Desarrollo **el mismo turno en que se compró**.
4. Fin de Turno: El turno pasa al siguiente jugador en el sentido de las agujas del reloj.

3.4. Puntos de Victoria y Cartas Especiales

Los jugadores buscan alcanzar **10 PV**.

A. Fuentes de PV

- **Poblado:** 1 PV (por cada uno)
- **Ciudad:** 2 PV (por cada una)
- **Carta de Desarrollo (PV):** 1 PV (por cada una, se mantienen ocultas hasta el final)
- **Gran Caballería:** 2 PV (ver sección B)

- **Gran Ruta Comercial:** 2 PV (ver sección B)

B. Cartas de Bonificación (2 PV)

Ambas bonificaciones son **otorgadas** y **robadas** dinámicamente:

- **Gran Caballería (2 PV):** Otorgada al jugador que juegue **3 o más** cartas de "Caballero" (del mazo de Desarrollo). Si otro jugador juega más cartas de Caballero, le **roba** esta bonificación.
- **Gran Ruta Comercial (2 PV):** Otorgada al jugador que tenga una **cadena contigua de al menos 5 Carreteras**. Si otro jugador construye una ruta más larga, le **roba** esta bonificación. (Las Carreteras no son bloqueadas por Poblados/Ciudades de otros jugadores, solo se rompen si la cadena es interrumpida).

3.5. Cartas de Desarrollo (Funcionalidad)

El mazo de Desarrollo contiene cartas con los siguientes efectos:

Tipo de Carta	Cantidad	Efecto (Acción en el Turno)
Caballero	14	Mover el **Ladrón** y **robar un recurso** de un jugador adyacente (similar a un 7, pero sin descarte de cartas). Contribuyen a la **Gran Caballería**.
Punto de Victoria (PV)	5	Se mantiene oculta. Se revela al alcanzar 10 PV o al final del juego. **1 PV**.
Progreso - Descubrimiento	2	Permite al jugador tomar **2 recursos a su elección** de la banca.
Progreso - Construcción de Carreteras	2	Permite construir **2 Carreteras gratuitamente** (debe cumplir reglas de colocación).
Progreso - Monopolio	2	El jugador **nombra un recurso**. Todos los demás jugadores deben entregarle **todas las cartas** de ese tipo de recurso que posean.

3.6. Condiciones de Victoria

El juego termina inmediatamente cuando un jugador alcanza **10 Puntos de Victoria** durante su turno. Este jugador es el ganador.

4. Interfaz gráfica

La interacción entre el usuario y la aplicación deberá ser mediante una interfaz gráfica intuitiva. Consistirá en una aplicación de escritorio utilizando JavaFX y se pondrá mucho énfasis y se evaluará como parte de la consigna su usabilidad. (*en el anexo I se explicarán algunas buenas prácticas para armar la interfaz gráfica de usuario o GU*I)

5. Herramientas

1. JDK (Java Development Kit): Versión 1.8 o superior.
2. JavaFX
3. JUnit 5 y Mockito: Frameworks de pruebas unitarias para Java.
4. IDE (Entorno de desarrollo integrado): Su uso es opcional y cada integrante del grupo puede utilizar uno distinto o incluso el editor de texto que más le guste. Lo importante es que el repositorio de las entregas no contenga ningún archivo de ningún IDE y que la construcción y ejecución de la aplicación sea totalmente independiente del entorno de desarrollo.
    a. Eclipse
    b. IntelliJ
    c. Netbeans
5. Herramienta de construcción: Se deberán incluir todos los archivos XML necesarios para la compilación y construcción automatizada de la aplicación. El informe deberá contener instrucciones acerca de los comandos necesarios (preferentemente también en el archivo README.md del repositorio). Puede usarse Maven o Apache Ant con Ivy.
6. Repositorio remoto: Todas las entregas deberán ser subidas a un repositorio único en GitHub para todo el grupo en donde quedarán registrados los aportes de cada miembro. El repositorio puede ser público o privado. En caso de ser privado debe agregarse al docente corrector como colaborador del repositorio.
7. Git: Herramienta de control de versiones
8. Herramienta de integración continua: Deberá estar configurada de manera tal que cada *commit* dispare la compilación, construcción y ejecución de las pruebas unitarias automáticamente. Algunas de las más populares son:
    a. Travis-CI
    b. Jenkins
    c. Circle-CI
    d. GitHub Actions (recomendado)

Se recomienda basarse en la estructura del proyecto base armado por la cátedra.

6. Entregables

Para cada entrega se deberá subir lo siguiente al repositorio:

1. Código fuente de la aplicación completa, incluyendo también: código de la prueba, archivos de recursos.
2. Script para compilación y ejecución (Ant o Maven).
3. Informe, acorde a lo especificado en este documento (en las primeras entregas se podrá incluir solamente un enlace a Overleaf o a Google Docs en donde confeccionen el informe e incluir el archivo PDF solamente en la entrega final).

No se deberá incluir ningún archivo compilado (formato .class) ni tampoco aquellos propios de algún IDE (por ejemplo .idea). Tampoco se deberá incluir archivos de diagramas UML propios de alguna herramienta. Todos los diagramas deben ser exportados como imágenes de manera tal que sea transparente la herramienta que hayan utilizado para crearlos.

7. Formas de entrega

Habrá 4 entregas formales que tendrán una calificación de APROBADO o NO APROBADO en el momento de la entrega. Además, se contará con una entrega 0 preliminar.

Aquel grupo que acumule 2 no aprobados, quedará automáticamente desaprobado con la consiguiente pérdida de regularidad en la materia de todos los integrantes del grupo. *En cada entrega se deberá incluir el informe actualizado*.

8. Evaluación

El día de cada entrega, cada ayudante convocará a los integrantes de su grupo, solicitará el informe correspondiente e iniciará la corrección mediante una entrevista grupal. Es imprescindible la presencia de todos los integrantes del grupo el día de cada corrección.

Se evaluará el trabajo grupal y a cada integrante en forma individual. El objetivo de esto es comprender la dinámica de trabajo del equipo y los roles que ha desempeñado cada integrante del grupo. Para que el alumno apruebe el trabajo práctico debe estar aprobado en los dos aspectos: grupal e individual (se revisarán los commits de cada integrante en el repositorio).

Dentro de los ítems a chequear el ayudante evaluará aspectos formales (como ser la forma de presentación del informe), aspectos funcionales: que se resuelva el problema planteado y aspectos operativos: que el TP funcione integrado.

9. Entregables para cada fecha de entrega

Cada entrega consta de las pruebas + el código que hace pasar dichas pruebas.

Cada entrega presupone que los equipos realizarán refactor de su código según crean conveniente.

Entrega 0 (Semana 12 del calendario)

- Planteo de modelo tentativo
- Diagrama de clases general
- Diagrama de secuencia para el caso donde se calcula el puntaje de un jugador basado en las cartas que se encuentran en el tablero.

Entrega 1 (Semana 13 del calendario)

Pruebas (sin interfaz gráfica):

- Verificar la correcta asignación aleatoria de hexágonos de **Terreno** y **Fichas de Número**.
- Verificar que la **Regla de Distancia** se aplique al colocar poblados iniciales.
- Verificar que los jugadores reciban los **recursos iniciales** correctos según el segundo poblado colocado.
- Verificar que el lanzamiento de dados genere un número válido (2-12).
- Verificar la producción correcta: 1 recurso por Poblado, 2 recursos por Ciudad, adyacentes al número lanzado.
- Verificar que el Terreno bajo el **Ladrón** no produzca recursos.
- Verificar que si un jugador tiene **más de 7 cartas**, descarte correctamente la mitad, redondeando hacia abajo, al lanzar un 7.
- Verificar que el jugador activo pueda mover el **Ladrón** y **robar una carta aleatoria** de un jugador adyacente a la nueva ubicación.

Entrega 2 (Semana 14 del calendario)

Pruebas (sin interfaz gráfica):

- Verificar el consumo de recursos y la correcta colocación de una **Carretera** (adyacencia).
- Verificar el consumo de recursos y la validación de la **Regla de Distancia** al construir un **Poblado**.
- Verificar el consumo de recursos al **mejorar** un Poblado a una **Ciudad** y el cambio de PV.
- Verificar que el **Comercio Marítimo (Banca)** aplique correctamente las tasas 4:1, 3:1 y 2:1.
- Verificar que el jugador pueda realizar un **intercambio interior** (de jugador a jugador).
- Verificar el consumo de recursos al comprar una **Carta de Desarrollo** y que esta pase al mazo **oculto**.
- Verificar que una carta comprada en el turno actual **no pueda jugarse** inmediatamente.
- Planteo inicial de interfaz gráfica (mockups/dibujos) mostrando el mapa del tablero, el inventario de recursos, y el marcador de PV.

Entrega 3 (Semana 15 del calendario)

- Modelo del juego terminado
- Interfaz gráfica inicial básica: comienzo del juego y visualización del tablero e interfaz de usuario básica.
- Modelo del manejo de turnos en el juego.

Entrega 4 (Semana 16 del calendario)

Trabajo Práctico completo funcionando, con interfaz gráfica final, **sonidos** e informe completo.

Tiempo total de desarrollo del trabajo práctico: 5 semanas

10. Informe

El informe deberá estar subdividido en las siguientes secciones:

Supuestos

Documentar todos los supuestos hechos sobre el enunciado. Asegurarse de validar con los docentes.

Diagramas de clases

Varios diagramas de clases, mostrando la relación estática entre las clases. Pueden agregar todo el texto necesario para aclarar y explicar su diseño de manera tal que el modelo logre comunicarse de manera efectiva.

Diagramas de secuencia

Varios diagramas de secuencia, mostrando la relación dinámica entre distintos objetos planteando una gran cantidad de escenarios que contemplen las secuencias más interesantes del modelo.

Diagrama de paquetes

Incluir un diagrama de paquetes UML para mostrar el acoplamiento de su trabajo.

Detalles de implementación

Deben detallar/explicar qué estrategias utilizaron para resolver todos los puntos más conflictivos del trabajo práctico. Justificar el uso de herencia vs. delegación, mencionar que principio de diseño aplicaron en qué caso y mencionar qué patrones de diseño fueron utilizados y por qué motivos.

IMPORTANTE

No describir el concepto de herencia, delegación, principio de diseño o patrón de diseño. Solo justificar su utilización.

Excepciones

Explicar las excepciones creadas, con qué fin fueron creadas y cómo y dónde se las atrapa explicando qué acciones se toman al respecto una vez capturadas.

Anexo 0: ¿Cómo empezar?

Requisitos, análisis

¿Cómo se empieza? La respuesta es lápiz y papel. No código!.

1. Entiendan el dominio del problema. Definir y utilizar un lenguaje común que todo el equipo entiende y comparte. Ej.: Si hablamos de “X entidad”, todos entienden que es algo ... Si los conceptos son ambiguos nunca podrán crear un modelo congruente.
2. Compartan entre el grupo, pidan opiniones y refinen la idea en papel antes de sentarse a programar.

Anexo I: Buenas prácticas en la interfaz gráfica

El objetivo de esta sección es recopilar algunas recomendaciones en la creación de interfaces gráficas de usuario o GUI. La idea no es exigir un diseño refinado y prolijo, sino que pueda ser usado por el grupo de docentes de Algo3 sin impedimentos “lo mejor posible”.

Prototipo

Venimos escribiendo código, integrales y UML todo el cuatrimestre. Me están pidiendo una interfaz gráfica. ¿Cómo se empieza? La respuesta es lápiz y papel. No código!.

1. Armen un dibujo o prototipo en papel (pueden usar google docs o cualquier herramienta también) de todas las “pantallas”. No tiene que ser perfecto.
2. Compartan entre el grupo, pidan opiniones y refinen la idea en papel antes de sentarse a programar.

JavaFX

Entender cómo funciona JavaFX es clave para implementar la GUI correctamente. No subestimen el tiempo que lleva implementar y modificar la UI o interfaz de usuario. Lean todo lo que ofrece y las buenas prácticas de la tecnología. Hay plugins específicos para el IDE, pero por más que usen herramientas WYSIWYG, siempre conviene entender la API para mejorar el código autogenerado que casi nunca es óptimo.

Recomendaciones visuales

Tamaño de elementos

- Se recomienda un tamaño de tipografía de al menos a 10 puntos al mayor contraste negro contra blanco para asegurar la legibilidad, o bien 12 puntos.
- Para los botones o elementos interactivos, el tamaño mínimo del área debería ser de 18x18.
- Extra: Los elementos más importantes deberían ser más grandes y estar en posiciones más accesibles (poner ejemplos)

Contraste

- Aseguren que el texto y los elementos tengan buen contraste y se puedan leer bien
- Se sugiere un contraste cercano a 3 entre textos y fondos para texto grande e imágenes.
- Herramientas para verificar contraste: https://colourcontrast.cc/ https://contrast-grid.eightshapes.com

Uso del color

- La recomendación es no utilizar más de 3 colores para la UI y los elementos
- En el enunciado se ejemplifica con una paleta accesible, pero pueden usar cualquiera para las fichas
- Accesibilidad: Si usan para las fichas rojo y verde, o verde y azul, aseguren que las personas con daltonismo puedan distinguirlas usando letras o símbolos sobre las mismas.
- Dudas eligiendo paletas? https://color.adobe.com

Tipografía

- No se recomienda usar más de 2 tipografías para toda la aplicación
- Asegurarse de que esas tipografías se exporten correctamente en en TP
- Evitar tipografías “artísticas” para texto, menú y botones, ya que dificultan la lectura

Recomendaciones de interacción

Manejo de errores

- No escalar excepciones a la GUI. Es un No absoluto. Enviar a la consola.
- Si muestran errores al usuario, el mensaje de error debe estar escrito sin jerga técnica y permitir al usuario continuar y entender lo que está pasando
- Siempre optar por validar y prevenir errores, a dejar que el usuario ejecute la acción y falle.

Confirmaciones

- Antes de cerrar o ejecutar cualquier operación terminal, una buena práctica es pedir confirmación al usuario (para el alcance del tp no sería necesario)

Visibilidad del estado y otros

- Mostrar el estado en el cual está el juego. Siempre debería estar accesible
- Permitir al usuario terminar o cerrar en cualquier momento